PAYMENT SYSTEM

Features, Flow & Developer Assignment

PhysioGuidance eLearning Platform

Roya Almustaqbal for Health Training

Document	Payment System — Features, Flow & Developer Assignment
Version	1.0
Date	March 02, 2026
Prepared By	Gamma Developers
Client	Roya Almustaqbal for Health Training
Classification	Confidential — Developer Internal

TABLE OF CONTENTS

1. Project Context & Current Status
2. Payment Gateways Overview
3. System Architecture
4. Database Schema
5. Complete Payment Flow
6. Refund Flow
7. Invoice Generation
8. Webhook Security
9. API Endpoints Reference
10. Jira Tickets & Build Order
11. Environment Variables & Credentials
12. Testing Strategy
13. NELC Compliance Checklist

1. PROJECT CONTEXT & CURRENT STATUS

Platform: PhysioGuidance — NELC-compliant eLearning for healthcare training

Client: Roya Almustaqbal for Health Training (Saudi Arabia)

Tech Stack: Next.js + TypeScript + Tailwind (Frontend) | NestJS + Drizzle ORM + PostgreSQL (Backend) | GCP (Infrastructure)

What's Already Built

Finance Manager Dashboard — Mock UI Complete

- FIN-001: KPI Cards (Revenue MTD, Net Revenue, AOV, Pending Invoices, Refund Rate)
- FIN-002: Revenue Summary Widget (breakdown by gateway: Tap/Moyasar vs BNPL Tamara/Tabby)
- FIN-003: Payments & Invoices Table (status badges, gateway icons, search, filters, pagination)
- FIN-004: Refunds Queue Widget (pending refunds, approve/reject, deduction rules display)
- FIN-005: Discounts & Coupons Widget (create codes, track usage, measure lift)

Learner Dashboard — Enrollment Flow UI Complete

- Course catalog → Enroll → Payment method selection → Confirmation page
- Free courses: immediate enrollment (no payment)
- Paid courses: payment gateway selection UI → redirect placeholder → success/failure pages
- NELC learner limits enforced in UI (40 long courses, 50 short courses)

What Does NOT Exist Yet — This Is Your Assignment

- No backend payment module (no controllers, services, or schemas)
- No gateway integration (Tap, Moyasar, Tamara, Tabby — none connected)
- No webhook handlers for payment status updates
- No invoice generation (PDF creation, numbering, storage)
- No refund processing logic
- No discount code validation at checkout

This is a ground-up backend build + frontend wiring.

2. PAYMENT GATEWAYS OVERVIEW

The platform supports two categories of payment processing:

Category 1 — Direct Payment (Tap / Moyasar)

Learner pays full amount immediately. Standard redirect-based or embedded checkout flows.

Feature	Tap Payments	Moyasar
Card Support	Visa, Mastercard, Amex	Visa, Mastercard
Mada (Saudi Debit)	Yes	Yes
Apple Pay	Yes	Yes
API Style	REST, JSON	REST, JSON
Auth Method	Bearer Token (sk_xxx)	HTTP Basic Auth
Webhook Events	charge.captured, charge.failed	payment_paid, payment_failed
Signature Verification	HMAC-SHA256 (hashstring header)	Shared secret comparison
Test/Sandbox Mode	Yes (sk_test_ prefix)	Yes (separate test keys)
PCI Compliance	PCI-DSS Level 1	PCI-DSS Compliant
Settlement	SAR direct	SAR direct

Category 2 — Buy Now Pay Later / BNPL (Tamara / Tabby)

Learner splits payment into installments (typically 3-4 payments). Platform receives full amount upfront from BNPL provider. Special deduction rules apply on refunds.

Feature	Tamara	Tabby
Installments	3 payments (split)	4 interest-free payments
Platform Receives	Full amount upfront	Full amount upfront
Refund Fee	~10% processing fee deducted	Per merchant agreement
API Style	REST, JSON	REST, JSON
Auth Method	API Token (Bearer)	Secret Key
Webhook Verification	Notification Token	HMAC signing key

Feature	Tamara	Tabby
Eligibility Check	Yes (pre-checkout)	Yes (pre-checkout)
Min/Max Order	Merchant-configurable	Merchant-configurable

Critical Architecture Decision — Gateway-Agnostic Pattern

Build the backend using a **Provider Pattern** (abstract interface + gateway adapters). The PaymentsService never calls a gateway directly — it uses a factory to get the right adapter based on gateway name. This means swapping or adding gateways requires only a new adapter file, not a system rewrite.

Client (Amna) has not confirmed final gateway selection. The contract mentions Paytabs/Moyasar/Tamara/Tabby, and Tap appears in dashboard specs. Build for all, let client decide at deployment.

3. SYSTEM ARCHITECTURE

Backend Module Structure (NestJS)

File	Purpose
payments.module.ts	Root module — registers all payment sub-modules
payments.controller.ts	REST endpoints for initiate, verify, callback, history
payments.service.ts	Orchestration logic — coordinates gateways, DB, enrollment
gateways/gateway.interface.ts	Abstract PaymentGateway interface (build FIRST)
gateways/tap.gateway.ts	Tap Payments adapter implementing interface
gateways/moyasar.gateway.ts	Moyasar adapter implementing interface
gateways/tamara.gateway.ts	Tamara BNPL adapter implementing interface
gateways/tabby.gateway.ts	Tabby BNPL adapter implementing interface
webhooks/webhooks.controller.ts	POST /api/webhooks/{gateway} — one per gateway
webhooks/webhooks.service.ts	Verify signature + process status update
invoices/invoices.service.ts	Create, list, download invoices
invoices/invoice-pdf.service.ts	PDF generation (bilingual AR/EN, QR code)
refunds/refunds.service.ts	Request, approve, reject, gateway refund call
discounts/discounts.service.ts	CRUD + validation at checkout

Gateway Interface (Build This First)

Every gateway adapter implements this interface. The PaymentsService calls the factory, never a gateway directly.

```
export interface PaymentGateway { name: string; // 'tap' | 'moyasar' | 'tamara' | 'tabby' createCharge(params: CreateChargeParams): Promise<ChargeResult>; verifyWebhook(headers: any, body: any): boolean; processWebhook(body: any): { chargeId: string; status: string; }; refund(params: RefundParams): Promise<RefundResult>; getCharge(chargeId: string): Promise<any>; }
```

Key Interfaces:

- **CreateChargeParams:** amount, currency (SAR), description, customerName, customerEmail, customerPhone, redirectUrl, webhookUrl, metadata (enrollmentId, courseId, userId)
- **ChargeResult:** gatewayChargeId, redirectUrl (where to send learner), status, rawResponse
- **RefundParams:** gatewayChargeId, amount, reason

- **RefundResult:** gatewayRefundId, status (processing/completed/failed), rawResponse

4. DATABASE SCHEMA

Table: payments

Column	Type	Notes
id	UUID PK	gen_random_uuid()
enrollmentId	UUID FK → enrollments	CASCADE, NOT NULL
userId	UUID FK → users	CASCADE, NOT NULL
courseId	UUID FK → courses	CASCADE, NOT NULL
gateway	VARCHAR(20)	CHECK: tap, moyasar, tamara, tabby
gatewayChargeId	VARCHAR(255)	ID returned by gateway
paymentMethod	VARCHAR(20)	card, mada, apple_pay, bnpl
amount	DECIMAL(10,2)	NOT NULL
currency	VARCHAR(3)	DEFAULT 'SAR'
status	VARCHAR(20)	pending / processing / paid / failed / refunded / partially_refunded
gatewayResponse	JSONB	Full raw response for audit
paidAt	TIMESTAMP	Set on successful payment
failedAt	TIMESTAMP	Set on failure
failureReason	VARCHAR(500)	Error message from gateway
metadata	JSONB	Installment plan, BNPL details
createdAt	TIMESTAMP	DEFAULT now()
updatedAt	TIMESTAMP	DEFAULT now()

Indexes: userId, courseId, status, gateway, createdAt

Table: invoices

Column	Type	Notes
id	UUID PK	gen_random_uuid()
invoiceNumber	VARCHAR(50) UNIQUE	Auto: INV-2026-00001

Column	Type	Notes
paymentId	UUID FK → payments	CASCADE, NOT NULL
userId	UUID FK → users	CASCADE, NOT NULL
courseId	UUID FK → courses	CASCADE, NOT NULL
subtotal	DECIMAL(10,2)	Before discount/tax
discountAmount	DECIMAL(10,2)	DEFAULT 0
taxAmount	DECIMAL(10,2)	DEFAULT 0
totalAmount	DECIMAL(10,2)	Final charged amount
status	VARCHAR(20)	issued / paid / overdue / cancelled
issuedAt / paidAt / dueDate	TIMESTAMP	Lifecycle timestamps
pdfUrl	VARCHAR(500)	Cloud Storage path
metadata	JSONB	Line items, buyer info snapshot

Indexes: invoiceNumber, userId, status

Table: refunds

Column	Type	Notes
id	UUID PK	gen_random_uuid()
paymentId	UUID FK → payments	CASCADE, NOT NULL
invoiceId	UUID FK → invoices	CASCADE
requestedBy	UUID FK → users	Learner or admin
processedBy	UUID FK → users	Finance manager
reason	VARCHAR(20)	course_issue / personal / technical / duplicate / other
reasonNote	TEXT	Free-text explanation
originalAmount	DECIMAL(10,2)	Full payment amount
deductionAmount	DECIMAL(10,2)	e.g., Tamara 10% fee
refundAmount	DECIMAL(10,2)	original - deduction
status	VARCHAR(20)	pending / approved / rejected / processing / completed / failed

Column	Type	Notes
gatewayRefundId	VARCHAR(255)	Refund ID from gateway
approvedAt / completedAt / rejectedAt	TIMESTAMP	Lifecycle timestamps
rejectionNote	TEXT	If rejected, reason provided

Indexes: paymentId, status

Table: discount_codes

Column	Type	Notes
id	UUID PK	gen_random_uuid()
code	VARCHAR(50) UNIQUE	e.g., SAVE20, LAUNCH50
type	VARCHAR(10)	percentage or fixed
value	DECIMAL(10,2)	15 = 15% off, or 50 = 50 SAR off
maxUses	INTEGER	null = unlimited
currentUses	INTEGER DEFAULT 0	Incremented on use
minOrderAmount	DECIMAL(10,2)	Minimum cart value to apply
courseId	UUID FK → courses	null = applies to all courses
validFrom / validUntil	TIMESTAMP	Active window
isActive	BOOLEAN DEFAULT true	Manual toggle
createdBy	UUID FK → users	Who created it

Table: discount_usage

Column	Type	Notes
id	UUID PK	gen_random_uuid()
discountId	UUID FK → discount_codes	CASCADE
userId	UUID FK → users	CASCADE
paymentId	UUID FK → payments	CASCADE
amountSaved	DECIMAL(10,2)	Actual discount applied
usedAt	TIMESTAMP	DEFAULT now()

Constraint: UNIQUE(discountId, userId, paymentId)

5. COMPLETE PAYMENT FLOW

End-to-end payment lifecycle from learner enrollment to course activation.

STEP 1 — Learner Clicks 'Enroll Now'

Frontend shows payment method selection screen: Card/Mada/Apple Pay (via Tap or Moyasar) or BNPL (Tamara/Tabby). Free courses skip payment entirely and create enrollment immediately.

STEP 2 — Frontend Calls POST /api/payments/initiate

Request body: { courseId, gateway, paymentMethod, discountCode (optional), locale: 'ar'|'en' }. Backend validates course exists, is Active, has seats available, learner not already enrolled, and discount code validity.

STEP 3 — Backend Creates Records + Calls Gateway

Calculate: subtotal - discount + tax = total. Create **payments** record (status: pending). Create **enrollment** record (status: pending_payment). Call gateway.createCharge() with amount, redirectUrl, webhookUrl. Store gatewayChargeId. Return redirect URL to frontend.

STEP 4 — Learner Redirected to Gateway Page

Tap/Moyasar: standard card entry page with Mada/Apple Pay options. Tamara/Tabby: BNPL approval flow where learner confirms installment plan. Learner completes or cancels payment on gateway's hosted page.

STEP 5 — Gateway Redirects Learner to Callback URL

GET /api/payments/callback?tap_id=xxx (varies by gateway). Backend looks up payment by gateway charge ID, checks current status. If already confirmed by webhook → redirect to success page. If not yet → show 'Processing...' and frontend polls POST /api/payments/verify/:paymentId.

STEP 6 — Gateway Sends Webhook (Async)

POST /api/webhooks/{gateway}. Backend: (1) Verify webhook signature — CRITICAL, prevents fake payments. (2) Extract charge ID and status. (3) Find matching payment record. (4) If PAID: update payment.status='paid', enrollment.status='active', generate invoice, send confirmation email. (5) If FAILED: update payment.status='failed', enrollment.status='cancelled', log audit event.

STEP 7 — Learner Sees Success — Course in My Courses

Success page shows: order summary, invoice download link, 'Go to Course' button. Course card appears in Learner Dashboard with progress bar at 0%. Calendar populates with upcoming sessions. Confirmation email sent with invoice PDF attached.

6. REFUND FLOW

STEP 1 — Refund Request Created

Learner requests via 'Request Refund' button in My Payments, or Finance Manager/Admin initiates on their behalf. POST /api/refunds with { paymentId, reason, reasonNote }.

STEP 2 — Appears in Finance Dashboard Queue

Refund shows in FIN-004 Refunds Queue Widget with status: PENDING. Finance Manager sees: original amount, gateway used, course name, learner info, reason.

STEP 3 — Finance Manager Reviews Deduction Rules

System auto-calculates deduction based on gateway: Tap/Moyasar = full refund (no processing fee). Tamara = 10% processing fee deducted. Tabby = per merchant agreement. Finance Manager sees net refund amount before approving.

STEP 4 — Finance Manager Approves or Rejects

APPROVE: Backend calls gateway.refund() with calculated net amount. Refund status → 'processing'. REJECT: Refund status → 'rejected' with rejectionNote. Learner notified via email.

STEP 5 — Gateway Processes Refund

Gateway sends webhook confirmation. Backend: update refund.status='completed'. Update payment.status='refunded' or 'partially_refunded'. Update enrollment.status='cancelled'. Send refund confirmation email to learner with amount breakdown.

Deduction Rules Reference

Gateway	Type	Refund Deduction	Learner Receives
Tap	Direct	0% — full refund	100% of original
Moyasar	Direct	0% — full refund	100% of original
Tamara	BNPL	~10% processing fee	~90% of original
Tabby	BNPL	Per merchant agreement	Varies

7. INVOICE GENERATION

Every successful payment generates an invoice automatically. Invoices must be bilingual (Arabic + English) and downloadable as PDF.

Invoice Contents

- **Branding:** Roya Almustaqbal for Health Training logo + Gamma Developers (if applicable)
- **Invoice Number:** Auto-incrementing format INV-2026-00001, INV-2026-00002, etc.
- **Dates:** Date issued, due date (for BNPL installments), payment date
- **Learner Info:** Full name, email, National ID (if available)
- **Course Info:** Course name (AR + EN), description, delivery mode
- **Line Items:** Subtotal, discount applied (code + amount), tax (if any), total
- **Payment Details:** Gateway used, payment method, transaction ID
- **QR Code:** Links to invoice verification URL (reuse certificate QR verification pattern)
- **Bilingual Layout:** Arabic (RTL) and English (LTR) on same invoice

Technical Implementation

- Use **pdfkit** (Node.js) or **@react-pdf/renderer** for server-side PDF generation
- Store generated PDFs in **GCP Cloud Storage** bucket
- Save Cloud Storage URL on invoice record (pdfUrl field)
- Invoice number counter: use a PostgreSQL sequence or max(invoiceNumber)+1 query with locking
- Trigger: on webhook confirmation of payment (not on initiation)

8. WEBHOOK SECURITY

Non-Negotiable — Signature Verification Required

Without webhook signature verification, anyone could POST a fake 'payment successful' to your webhook endpoint and get free enrollment. This is a critical security requirement.

If verification fails → return 401, DO NOT process, log the attempt for security audit.

Gateway	Verification Method	Implementation
Tap	HMAC-SHA256	Verify hashstring header against API secret key
Moyasar	Shared Secret	Compare signature in payload against configured webhook secret
Tamara	Notification Token	Compare token in headers against configured notification secret
Tabby	HMAC Signing Key	Verify HMAC signature using webhook signing key

Webhook Processing Rules

- **Idempotency:** Process each webhook only once. Check if payment status already updated before processing. Use gatewayChargeId as dedup key.
- **Response:** Return 200 OK immediately after signature verification. Process payment asynchronously (queue via BullMQ/Redis) to avoid gateway timeouts.
- **Retry Handling:** Gateways retry failed webhooks. Idempotency check prevents double-processing.
- **Logging:** Log all webhook payloads (sanitized) in gatewayResponse JSONB field for NELC audit trail.
- **Timeout:** Return response within 5 seconds. Queue heavy operations (invoice PDF, email) for background processing.

9. API ENDPOINTS REFERENCE

Checkout Flow (Learner-Facing)

Method	Endpoint	Description
POST	/api/payments/initiate	Start payment → returns redirect URL
GET	/api/payments/callback	Gateway redirects here after payment
POST	/api/payments/verify/:id	Frontend polls to check payment status
GET	/api/payments/my-payments	Learner's payment history

Webhook Endpoints (Gateway Calls These)

Method	Endpoint	Description
POST	/api/webhooks/tap	Tap payment status webhook
POST	/api/webhooks/moyasar	Moyasar payment status webhook
POST	/api/webhooks/tamara	Tamara BNPL status webhook
POST	/api/webhooks/tabby	Tabby BNPL status webhook

Invoice Endpoints

Method	Endpoint	Description
GET	/api/invoices	List invoices (learner: own, finance: all)
GET	/api/invoices/:id	Invoice detail
GET	/api/invoices/:id/pdf	Download invoice PDF
POST	/api/invoices/:id/resend	Resend invoice email

Refund Endpoints (Finance Manager)

Method	Endpoint	Description
GET	/api/refunds	List refund requests (?status=pending)
POST	/api/refunds	Create refund request (learner or admin)

Method	Endpoint	Description
PATCH	/api/refunds/:id/approve	Approve → triggers gateway refund
PATCH	/api/refunds/:id/reject	Reject with reason

Discount Endpoints (Finance / Admin)

Method	Endpoint	Description
GET	/api/discounts	List discount codes
POST	/api/discounts	Create new code
PATCH	/api/discounts/:id	Update code
DELETE	/api/discounts/:id	Deactivate code
POST	/api/discounts/validate	Validate at checkout { code, courseId }

Finance Dashboard Endpoints

Method	Endpoint	Description
GET	/api/finance/summary	Revenue, refunds, AOV, trends (date filters)
GET	/api/finance/revenue-by-source	Breakdown by gateway, method, course
GET	/api/finance/payment-status	Paid/pending/failed counts + amounts
GET	/api/finance/refund-rate	Refund rate by course and gateway
GET	/api/finance/invoice-aging	Overdue invoice count and total

10. JIRA TICKETS & BUILD ORDER

Epic: PAY — Payment System Integration

Total Estimate: ~23 dev-days (~4.5 weeks for one full-stack developer)

Ticket	Title	Priority	Blocked By	Est
PAY-1	DB schemas + migrations (payments, invoices, refunds, discounts)	Critical	None	1d
PAY-2	PaymentGateway interface + MockGateway + factory	Critical	None	0.5d
PAY-3	PaymentsService — initiate, verify, callback logic	Critical	PAY-1,2	2d
PAY-4	Webhook controller + signature verification + processing	Critical	PAY-3	1.5d
PAY-5	Enrollment integration — payment→pending, webhook→active	Critical	PAY-3,4	1d
PAY-6	Invoice service + PDF generation (bilingual, QR)	High	PAY-1	2d
PAY-7	Refund service — request, approve, reject, gateway call	High	PAY-3	1.5d
PAY-8	Discount codes CRUD + checkout validation	Medium	PAY-1	1d
PAY-9	Finance Dashboard API endpoints (summary, revenue, aging)	High	PAY-1	1.5d
PAY-10	Tap gateway adapter (real API calls)	High	PAY-2	1d
PAY-11	Moyasar gateway adapter	High	PAY-2	1d
PAY-12	Tamara BNPL adapter	Medium	PAY-2	1.5d
PAY-13	Tabby BNPL adapter	Medium	PAY-2	1.5d
PAY-14	Frontend — Checkout flow (select→redirect→callback→result)	Critical	PAY-3	2d
PAY-15	Frontend — Wire Finance Dashboard widgets to real endpoints	High	PAY-9	2d
PAY-16	Frontend — Invoice download + resend	Medium	PAY-6	0.5d

Ticket	Title	Priority	Blocked By	Est
PAY-17	Frontend — Refund request (learner) + queue (finance)	High	PAY-7	1.5d
PAY-18	Frontend — Discount code input + management UI	Medium	PAY-8	1d
PAY-19	Email notifications (confirmation, invoice, refund status)	Medium	PAY-5,6	1d
PAY-20	Audit logging for all payment events (NELC)	High	PAY-3	0.5d

Recommended Build Order

Week	Tickets	Focus
Week 1	PAY-1, PAY-2, PAY-3, PAY-4, PAY-5	Core backend — schemas, interface, payment flow, webhooks, enrollment link
Week 2	PAY-6, PAY-7, PAY-8, PAY-9, PAY-20	Invoices, refunds, discounts, finance APIs, audit logs
Week 3	PAY-14, PAY-15, PAY-17	Frontend — checkout flow, finance dashboard wiring, refund UI
Week 4	PAY-10-13, PAY-16, PAY-18, PAY-19	Real gateway adapters (needs credentials), remaining frontend, emails

11. ENVIRONMENT VARIABLES & CREDENTIALS

Required from Client (Amna)

Gateway	Credentials Needed	Where to Get
Tap	Secret Key (sk_live_xxx), Public Key (pk_live_xxx)	dashboard.tap.company
Moyasar	Secret Key, Publishable Key	dashboard.moyasar.com
Tamara	API Token, Notification Token	Tamara merchant portal
Tabby	Secret Key, Public Key, Webhook Signing Key	Tabby merchant dashboard

Security — Credential Handling

- Share credentials via secure channel ONLY (1Password, Bitwarden Send, encrypted email)
- NEVER share over Upwork chat, Slack, or plain text email
- Store in GCP Secret Manager for production (not .env files)
- Use test/sandbox keys during development — request test keys FIRST
- Rotate keys if any suspected exposure

Environment Variables Template

```
# Tap Payments TAP_SECRET_KEY=sk_test_xxx TAP_PUBLIC_KEY=pk_test_xxx TAP_WEBHOOK_SECRET=whsec_xxx # Moyasar MOYASAR_SECRET_KEY=sk_test_xxx MOYASAR_PUBLISHABLE_KEY=pk_test_xxx MOYASAR_WEBHOOK_SECRET=xxx # Tamara TAMARA_API_TOKEN=xxx TAMARA_NOTIFICATION_TOKEN=xxx TAMARA_API_URL=https://api-sandbox.tamara.co # Tabby TABBY_SECRET_KEY=sk_test_xxx TABBY_PUBLIC_KEY=pk_test_xxx TABBY_WEBHOOK_SECRET=xxx # General PAYMENT_CALLBACK_URL=https://physioguidance.com/payment/callback PAYMENT_WEBHOOK_BASE_URL=https://physioguidance.com/api/webhooks
```

12. TESTING STRATEGY

Phase	Description	Timeline
Phase 1: Local (MockGateway)	Use MockGateway that returns configurable success/failure. Test entire flow end-to-end without real money. No credentials needed.	Weeks 1-2
Phase 2: Sandbox	Once test credentials arrive, use gateway sandbox modes with test card numbers (each gateway provides these). Test real redirect flows, webhook delivery, signature verification.	Week 3
Phase 3: Webhook Testing	Use ngrok or Cloudflare Tunnel to expose local webhook endpoints to the internet so gateways can reach them during sandbox testing.	Week 3
Phase 4: Production Smoke	Swap test keys for live keys. Make one small real payment (1 SAR) to verify full end-to-end. Confirm webhook receipt, enrollment activation, invoice generation, email delivery.	Week 4

What Developer Can Build WITHOUT Credentials

Build Immediately — No Credentials Needed

- All database schemas and Drizzle migrations
- Complete PaymentGateway interface + factory pattern
- All API endpoints (controllers, services, DTOs, validation)
- MockGateway implementation (simulates success/failure)
- Invoice PDF generation service
- Refund workflow logic (request → approve → process)
- Discount code CRUD + checkout validation
- All Finance Dashboard API endpoints
- Frontend checkout flow UI (method selection → redirect → callback → result)
- Frontend wiring — connect Finance Dashboard mock widgets to real endpoints
- Audit logging for all payment events

Needs Credentials — Wire Last

- Actual Tap/Moyasar/Tamara/Tabby gateway adapter HTTP calls
- Webhook signature verification (needs real secrets)
- Production redirect URLs and webhook URLs

13. NELC COMPLIANCE CHECKLIST — PAYMENTS

All payment features must meet NELC and PDP compliance requirements:

Requirement	Implementation	Related Tickets
Audit Trail	Every payment event (initiate, success, failure, refund) logged with immutable timestamp, user ID, IP address, and action detail	PAY-20
Data Protection (PDP)	Payment data encrypted at rest (PostgreSQL TDE) and in transit (TLS). No raw card data stored — handled by gateway. Gateway response stored in JSONB for audit only.	PAY-1
Export Evidence	All payment reports exportable as CSV/PDF with timestamp, filters used, and user context for NELC audit evidence bundles	PAY-9
Bilingual Support	Invoice PDFs in Arabic + English. Payment flow UI supports RTL/LTR toggle. Error messages localized.	PAY-6, PAY-14
Accessibility (WCAG 2.1 AA)	Checkout flow keyboard-navigable, screen reader compatible. Color contrast ratios met. Focus indicators visible.	PAY-14
Refund Policy	Published refund policy accessible to all users. Deduction rules transparent before learner confirms. Policy linked in platform footer.	PAY-7
Financial Reporting	Revenue, refund, and invoice reports available for NELC auditors with date range filters and drill-down capability	PAY-9, PAY-15
RBAC Enforcement	Learners: own payments only. Finance Manager: all payments + refund approval. Admin: full access + policy config. No cross-role data leaks.	All tickets

Next Steps

1. **1.** Developer begins with PAY-1 (schemas) and PAY-2 (gateway interface) — no blockers.
2. **2.** Request test/sandbox API keys from Amna for all confirmed gateways.
3. **3.** Share credentials via secure channel only (1Password or Bitwarden Send).
4. **4.** Weeks 1-3 proceed with MockGateway while awaiting real credentials.
5. **5.** Week 4: wire real gateway adapters, sandbox testing, then production smoke test.

Estimated Total: ~23 dev-days | ~4.5 weeks (1 full-stack developer)